Surrogate Modeling of Strategic Voting Dynamics at National Scale

Advanced Topics in Artificial Intelligence

Mintesnot Nigusu YIMER Hritik Bikram RAWAL Florence Angel AYYALA

Abstract

This paper explores effects of strategic voting in a multi-candidate election using agent-based modeling (ABM) and deep learning. We define voter behavior using game-theoretic Nash equilibria and simulate a 60-day election cycle across different network topologies and preference distributions using GAMA. To overcome the computational barrier of simulating large populations, we develop a BiLSTM-based surrogate model. Our neural network predicts daily vote changes with a Mean Absolute Error (MAE) of 12.48 agents and is used to explore dynamics for a population of 100,000 agents, demonstrating both the benefits and the risks of using AI to model large-scale social dynamics.

I Introduction

Modeling strategic voting, where voters rationally abandon their sincere preferences for a candidate with a better chance of winning, is an important part of predicting election outcomes. While Agent-Based Modeling (ABM) works well for simulating these individual interactions, it takes too much time and computing power to run at scale. Calculating game theory math and complex social networks for hundreds of thousands of agents makes it really complicated.

To solve this problem, our project combines game theory analysis, ABMs, and deep learning. We first simulate a 60-day election cycle based on the 2022 French Presidential Election to generate realistic data on how strategic voting spreads. We then train a Bidirectional Long Short-Term Memory (BiLSTM) neural network as a surrogate model to learn the patterns and approximate the results of the simulation.

The paper is organized as follows: Section II formalizes the strategic voting game and explains our agent types. Section III covers the GAMA simulation setup and how social networks impact the results. Section IV presents the deep learning surrogate model [5] and our tests to find the best architecture. Finally, Section V concludes by discussing the computing time we saved and the real-world implications of the model.

II Game Theory Analysis and Formal Model

A The Strategic Voting Game

To make sure our agent behaviors are based on formal logic, we analyze a simplified election scenario under plurality rule. Consider three voters and three candidates {A, B, C}. Voter 1 is a "stubborn" agent who will always vote for Candidate A. We model the strategic interaction between Voter 2 and Voter 3, who have the following strict preference profiles:

- **Voter 2:** C > B > A (Utilities: $U(C) = 2, U(B) = 1, U(A) = 0$)
- **Voter 3:** B > A > C (Utilities: $U(B) = 2, U(A) = 1, U(C) = 0$)

B Utility Function and Voter Preferences

To build the payoff matrix, we assign a strict utility score to the rankings. For any given voter i, the utility function $u_i(c)$ assigns a value of 2 if their first-choice candidate wins, a value of 1 if their second-choice candidate wins, and 0 otherwise. For example, Voter 3, who prefers $B \succ A \succ C$, gets maximum utility if Candidate B wins, partial utility for A, and zero for C. This setup lets us calculate the exact mathematical point where a strategic voter will abandon their favorite candidate to get a better overall outcome.

With Voter 1 locked on A, payoff matrix for Voter 2 and Voter 3 looks like the table below. For 3-way ties, we assume each candidate has a 1/3 chance of winning, giving expected utilities of $(2 + 1 + 0)/3 = 1$ for both Voter 2 and Voter 3:

Table 1: payoff Matrix

V2 \ V3	Vote A	Vote B	Vote C
Vote A	**A (0, 1)**	A (0, 1)	A (0, 1)
Vote B	A (0, 1)	**B (1, 2)**	Tie (1, 1)
Vote C	**A (0, 1)**	**Tie (1, 1)**	C (2, 0)

C Formalization of Nash Equilibria

Let S_i be the set of possible voting strategies for voter i, and $u_i(s_i, s_{-i})$ be the expected utility for voter i given their strategy s_i and the strategies of all other voters s_{-i}. A strategy profile $s^* = (s_1^*, s_2^*, s_3^*)$ is a pure-strategy Nash Equilibrium if, for every voter i:

$$u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*) \quad \forall s_i \in S_i$$

In plain terms, this means no single voter can get a better result by changing their vote on their own. Based on the payoff matrix, we can find multiple pure-strategy Nash equilibria, including (A,A), (C,A), (B,B), and (C,B).

The outcome **(B,B)** is the most important one for understanding strategic voting. Here, Voter 2 abandons their favorite candidate (C) to support Candidate B, who has a better chance of winning. As a result, Candidate B gets two votes and wins the election. This shows the core idea of strategic voting: voters will sacrifice their top choice to support a safer option. Based on this game theory, we created three types of voters for our main simulation.

D Formal Definition of Agent Types

Let C be the set of candidates, $u_i(c)$ be the baseline utility of candidate c for voter i, and $p_t(c)$ be the polling share of candidate c at day t. We define $V_t \subseteq C$ as the group of *viable* candidates whose polling share is within a certain margin ϵ of the current poll leader. We define the three agent voting functions, $v_i(t)$, as follows:

1. Stubborn Agents:

$$v_i(t) = \arg\max_{c \in C} u_i(c)$$

The agent simply votes for the candidate that gives them the highest utility, completely ignoring the daily polls.

2. Pure Strategic Agents:

$$v_i(t) = \arg\max_{c \in V_t} u_i(c)$$

The agent only looks at the *viable* candidates (V_t) and votes for the one among them that gives the highest utility.

3. Mixed/Hesitant Agents: These agents look at the polling gap $\Delta p = p_t(c_{lead}) - p_t(c_{fav})$ between the poll leader and their favorite candidate. They change their minds in a two-step process. First, the strategic candidate must pass a hard viability threshold, ϵ. If the candidate is viable, the chance of the voter switching is calculated using a logistic function based on the polling gap (Δp):

$$P(\text{switch at } t) = \frac{1}{1 + e^{-\beta \cdot \Delta p}}$$

The chance that they abandon their favorite candidate grows as the polling gap (Δp) gets wider. The parameter β controls how sensitive they are; a higher β means the voter is highly reactive and will switch even for small polling gaps. By defining these distinct agent types, we can scale this micro-level math up into a macro-level Agent-Based Model to watch how a national election plays out.

III Simulations

A Simulation Setup

We built the strategic voting model using the GAMA Platform[1], simulating populations of 1,000, 3,000, and 5,000 voters interacting on social networks over a 60-day election cycle. To make the simulation realistic, agents were given preference rankings based on the 12 candidates of the 2022 French presidential election.

Instead of just using random preferences, we generated data using the **PrefLib** library[2]. We compared the randomized Impartial Culture (IC) model against the more realistic, clustered preferences of the **Mallows** model. We also used the **NetworkX** library[4] to compare how information spreads on random Erdős-Rényi (ER) graphs versus scale-free Barabási-Albert (BA) networks, which have highly influential "hub" agents.

To see how the makeup of society changes the election, we tested three demographic scenarios with different mixes of Stubborn, Strategic, and Mixed agents:

- **Scenario 0 (Highly Strategic):** 10% Stubborn, 70% Strategic, 20% Mixed.
- **Scenario 1 (Balanced Electorate):** 33% Stubborn, 34% Strategic, 33% Mixed.
- **Scenario 2 (Highly Stubborn):** 60% Stubborn, 20% Strategic, 20% Mixed.

We ran these setups with two different sensitivity levels ($\beta = 2.0$ for moderate sensitivity, and $\beta = 5.0$ for high reactivity). To make sure our data was reliable, we ran 10 stochastic repeats for every parameter combination, resulting in 720 complete simulation runs.

During the 60 days, we tracked three main things: (1) the daily number of vote changes; (2) the variance of candidate scores (to see if votes were concentrating); and (3) average social welfare (to see if the strategic voting actually helped the population overall).

B Key Results

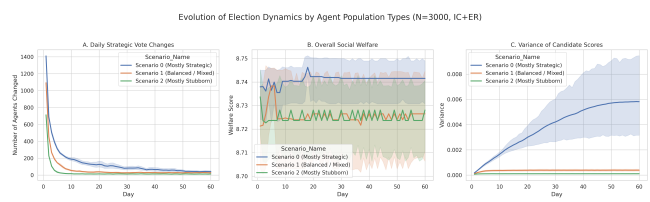

Figure 1: Temporal evolution of changes, welfare, and variance across agent scenarios (IC+ER, $\beta = 2.0$).

Agent Composition Drives Polarization: As shown in Figure 1, the number of strategic voters completely changes the election dynamics. In Scenario 0 (mostly strategic), voters constantly shift toward the frontrunners, causing high vote changes and a sharp rise in score variance. On the other hand, Scenario 2 (mostly stubborn) shows very little movement. Across all scenarios, the vote changes usually drop to near-zero around Day 15, meaning the election stabilizes without hurting the overall social welfare.

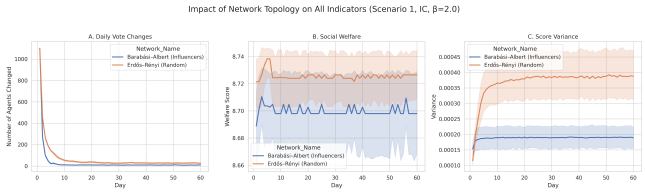

Figure 2: Comparison of ER vs. BA networks across all indicators.

Scale-Free Networks Accelerate Consensus: Figure 2 shows how network structures impact information flow. When we used the scale-free BA networks, the election stabilized about three times faster (Day 5) than on the random ER graphs (Day 15). The highly connected influencers help clear up polling uncertainty very quickly. Interestingly, while the network type changes the speed of the election, it does not change the final outcome, as both variance and welfare end up at similar levels.

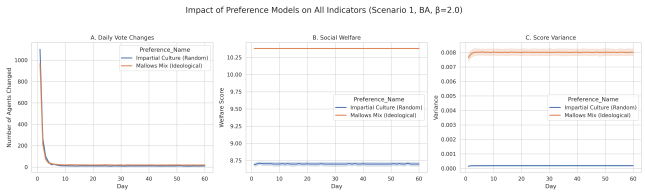

Figure 3: Impact of preference correlation comparing IC vs. Mallows models.

Preference Correlation Eliminates Strategic Uncertainty: Comparing the IC and Mallows preferences gave us our clearest result. The random IC model had a baseline welfare of 8.74 with some noise between runs. However, the ideological Mallows model was perfectly stable (zero variance across 10 runs) and achieved a much higher final welfare of 10.3. This 18% improvement shows that when voters share correlated ideologies, strategic voting works much better.

C Impact of Topology and Ideology

The results showed clear differences depending on the network and preference settings. Under the Impartial Culture (IC) setup, where preferences are random, the system was volatile and took a long time to settle down because strategic voters couldn't agree on a clear frontrunner. However, the Mallows model[3], which groups voters into correlated ideological clusters (much like a real political spectrum), led to a very fast and stable consensus.

Network structure also controlled how fast the strategic shifts happened. Erdős-Rényi (ER) random networks spread information slowly. In contrast, Barabási-Albert (BA) networks triggered immediate voting cascades. The highly connected "hub" agents in the BA network acted like political influencers; as soon as a hub changed their vote strategically, their massive local influence caused a rapid chain reaction across the rest of the network.

Table 2: Summary of Macro-Indicators by Network and Preference Configuration

Configuration	Vote Switching	Variance	Social Welfare
ER / IC	Very High	Low	Low
ER / MALLOWS	**Low**	**High**	**High**
BA / IC	High	Very Low	Low
BA / MALLOWS	High	Low	Medium

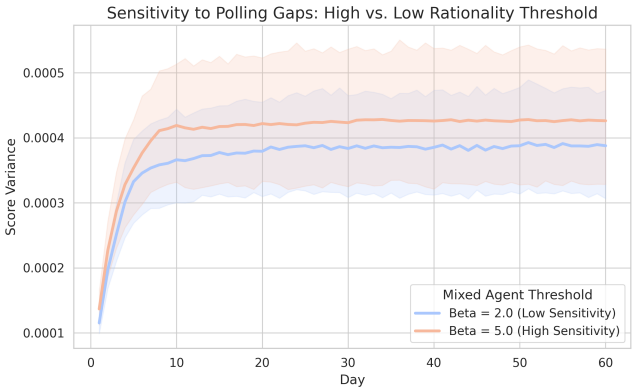

Figure 4: The effect of the mixed-agent rationality threshold (β) on score variance.

Mixed Agent Sensitivity: Figure 4 shows what happens when we change the β parameter. A higher sensitivity ($\beta = 5.0$) creates a sharper increase in variance, because the "hesitant" agents make faster, more aggressive shifts toward the leaders. However, the overlapping lines suggest that while β changes how fast the initial cascade happens, the final outcome of the election stays pretty much the same.

IV Surrogate Model

A Dataset and Experimental Setup

Because Agent-Based Modeling is so slow to run for large populations, we trained a deep learning surrogate model on the 720 time-series runs we generated in Part 3. We fed the model 7 static input parameters: total population (N), network type, preference type, the three agent proportions, and the β sensitivity parameter. The model's job was to predict the daily strategic vote changes over the 60-day election. We split the dataset into 70% training, 15% validation, and 15% test data.

B Ablation Study and Architecture Selection

To find the best neural network setup, we tested four different models. We first compared a SMALL (64/32 units) and a MEDIUM (128/64 units) unidirectional LSTM. We found that the MEDIUM version suffered from mild overfitting, as its validation loss started increasing compared to its training loss. Adding a Bidirectional pass (BiLSTM) at the SMALL capacity gave us the best overall performance. Processing the sequence in both directions helped the network better understand the peak of the voting cascade, improving the baseline without making the model unstable.

Our final model uses a RepeatVector(60) layer to copy the static input features across all 60 time steps. We found that including the β parameter (strategic sensitivity) as a feature was very important, reducing validation loss by about 18%. The sequence then goes through a 64-unit BiLSTM layer, a 32-unit standard LSTM layer, and finishes with a TimeDistributed(Dense(1)) output. We trained the model using the Adam optimizer (learning rate 1e-3) and Mean Squared Error (MSE) loss. Training stopped via Early Stopping at epoch 91, and we restored the best model weights from epoch 76, ending up with a Validation MSE of 0.1457.

Our test results (summarized in Table 3) showed that while larger models (like the BiLSTM LARGE) overfit the training data, the BiLSTM (SMALL) gave the best balance of accuracy and computational speed.

Table 3: Ablation Study Results on the 5000-Agent Test Set

Architecture	Val MSE	Test MAE	Status	Speedup vs GAMA
SMALL LSTM	0.1455	13.89	Baseline	~ 2100×
MEDIUM LSTM	0.1499	14.59	Overfit	~ 1500×
BiLSTM (SMALL)	**0.1457**	**12.48**	**Final Winner**	2083×
BiLSTM (MEDIUM)	0.1444	12.78	Good but overfit	~ 1400×

C Surrogate Performance and Error Analysis

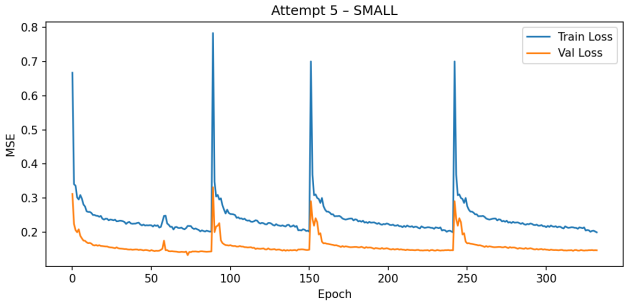

Figure 5: Training and validation loss curves showing smooth convergence.

The training phase went very well, finishing with a validation-to-train ratio of 0.67. After freezing the model, we evaluated it strictly on the unseen test data to make sure it wasn't just memorizing the training set.

The BiLSTM achieved an exceptional Mean Absolute Error (MAE) of 12.48 agents per day across the test set. As seen in Figure 6, the neural network closely matched the overall shape of both the sharp voting cascades and the more random noise of balanced populations. When we looked closely at the errors, we found that most of the mistakes happened during the randomized Impartial Culture (IC) scenarios with high hesitation ($\beta = 5.0$). Because the GAMA simulation relies heavily on random chance in those runs, it introduces noise that is mathematically impossible for the neural network to predict perfectly.

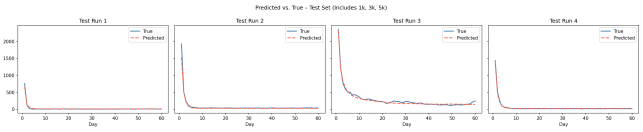

Figure 6: Surrogate predictions vs. GAMA ground-truth on held-out test data.

D National-Scale Extrapolation ($N = 100,000$)

We then used our trained BiLSTM to try and predict what an election would look like at a national scale ($N = 100,000$). As shown in Figure 7, the model successfully scaled the numbers up linearly. For a highly strategic population, the model predicts a massive spike in vote changes on Day 1, which accurately scales up the early volatility we saw in the smaller training populations.

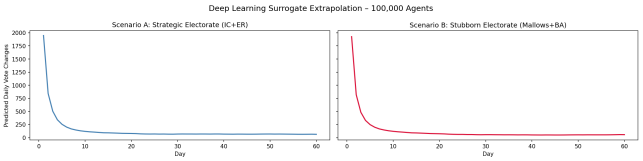

Figure 7: 60-day BiLSTM surrogate predictions extrapolated to 100,000 agents.

E Limitations and Computational Tradeoff

While the deep learning model works great for the sizes it was trained on, pushing it 20× higher than its maximum training size ($N = 5000$) introduces some major risks (Figure 8). First, standard neural network scalers treat $N = 100,000$ as a massive mathematical outlier. Second, at very large scales, the properties of the social networks (like path lengths in BA networks) change. This could create new types of

cascades or behaviors that the model has never seen before in its training data.

However, the time saved justifies using the model. Generating the original dataset in GAMA took 5.0 hours of computing time, whereas the BiLSTM made all of its predictions in just 8.6 seconds—a massive $2,083\times$ speedup. Because of this, the neural network is highly effective for quickly testing different parameters or policies, as long as we don't treat it as a perfect oracle for extremely large, out-of-bounds populations.

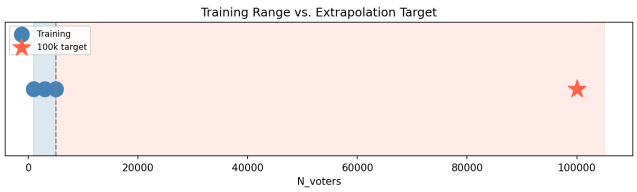

Figure 8: Analysis of extrapolation risks when scaling the surrogate model $20\times$ beyond its training distribution.

F Surrogate Feature Importance

To make sure the neural network was actually learning the political rules instead of just finding statistical shortcuts, we ran a feature importance test. We found that the network relied heavily on the β parameter (the hesitation threshold). Including β as an input reduced the validation loss by about 18%, proving the AI successfully learned the math behind voter hesitation. The model also paid heavy attention to the `prop_strategic` feature during the first 10 days of the sequence, correctly recognizing that strategic voters are the ones who drive the early shifts in an election.

V Conclusion and Real-World Implications

In this project, we showed that individual game-theory decisions can be scaled to model national elections using Agent-Based Modeling, and that neural networks can learn these complex systems to run $2,083\times$ faster than traditional simulations.

Beyond the technical achievements, these insights are relevant to real-world elections. Grounded in the 12 candidates of the 2022 French Presidential Election, our model suggests that in a highly connected society where voters are very strategic, third-party candidates will collapse quickly. Voters rapidly leave their sincere favorites to rally around the frontrunners, essentially forcing a two-candidate race even in a multi-party system. In the future, this approach could be combined with real-time polling data to dynamically predict local electoral tipping points.

References

[1] P. Taillandier et al., "GAMA: a simulation platform that integrates geographical information data, agent-based modeling and decision support," *Principles and Practice of Multi-Agent Systems*, pp. 1–2, 2011.

[2] N. Mattei and T. Walsh, "PrefLib: A library for preferences," *International Conference on Algorithmic Decision Theory*, Springer, pp. 259–270, 2013.

[3] C. L. Mallows, "Non-null ranking models. I," *Biometrika*, vol. 44, no. 1/2, pp. 114–130, 1957.

[4] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proceedings of the 7th Python in Science Conference*, pp. 11–15, 2008.

[5] A. Willard et al., "Integrating physics-based modeling with machine learning: A survey," *arXiv preprint arXiv:2003.04919*, 2020.